

Prompt Engineering Playbook

Aura — Autonomous Utility & Robotics Agent

Edwin Redhead & Team

AI for the Real World Hack — 4-hour build, 4-person team

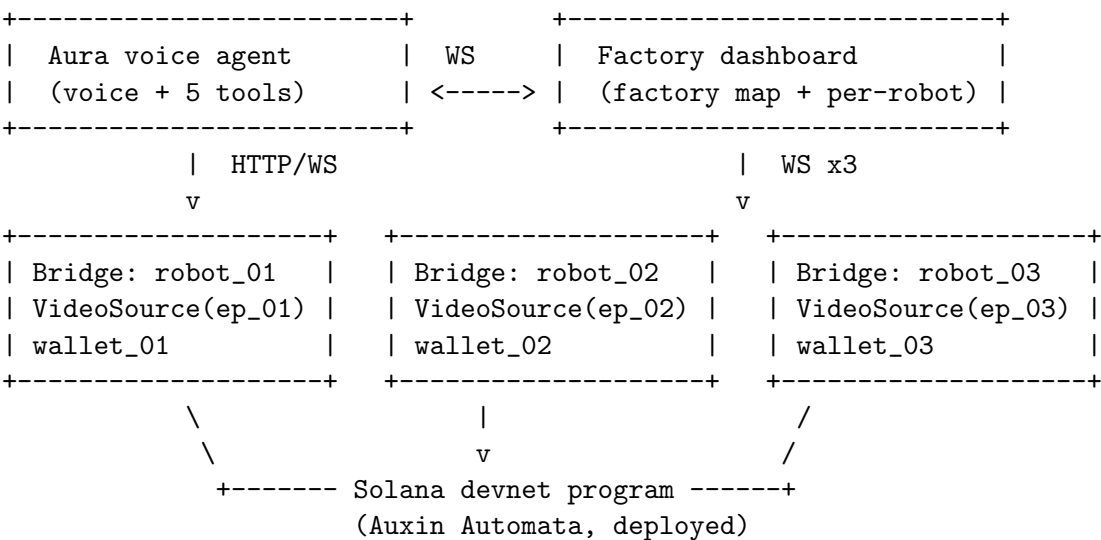
Abstract

Master prompts for building Aura, a voice-native operations agent for a fleet of robots, in a 4-hour hackathon sprint with a 4-person team. The existing Auxin Automata stack (Solana program on devnet, Python bridge, Gemini oracle, treasury agent, weasyprint invoicing) is treated as immutable infrastructure, with one new addition: a **VideoSource** that replays real Franka Emika Panda VLA episodes as live telemetry. Three bridge instances run in parallel, each representing one robot in a virtual factory, each with its own Solana wallet on devnet. A new factory-floor dashboard provides click-through navigation between robots. Four tracks run in parallel; sync points at 12:30, 13:30, 14:30, 15:00, 15:30. Each prompt is scoped tightly to one developer-hour or less.

Project Summary

Aura is a voice-controlled operations agent for an autonomous robot fleet. An operator surveys a factory-floor dashboard, clicks the robot they want to focus on, and speaks. Aura understands, queries the fleet, optionally pays for cloud inference from the selected robot’s own Solana wallet (with verbal confirmation), logs every action on chain, and replies in voice via ElevenLabs. The fleet is three real Franka Emika Panda episodes from a VLA dataset, replayed as live telemetry — three robots, three wallets, three independent agents on devnet. Pitch in one sentence: *Aura is the voice interface for a robot fleet where every machine has its own wallet and every action has an immutable receipt.*

Architecture at a Glance



The new monorepo is `aura/` with three workspaces: `aura-agent/` (voice + tools), `aura-dashboard/` (Next.js), `aura-fleet/` (multi-bridge runner + VideoSource). Auxin is referenced as a path dependency. No code in `Auxin_Automata/` is modified except adding the **VideoSource** class to the SDK.

Team & Tracks

Track	Owner	Scope
T1	Person 1	Aura voice agent. Voice loop, five fleet-aware tools, confirmation flow, system prompt. <code>aura-agent/</code> .
T2	Person 2	Fleet pipeline. <code>VideoSource</code> for VLA episodes, multi-bridge runner, fleet HTTP aggregator, anomaly injector. <code>aura-fleet/</code> + <code>Auxin_Automata/sdk/sources/video.py</code> .
T3	Person 3	Factory-floor dashboard. New Next.js app: factory map, click-through per-robot view, video feeds, Aura transcript panel. <code>aura-dashboard/</code> .
T4	Person 4	Robot identity + pitch. On-chain registration of three robots, metadata JSONs, identity tile, pitch script, run sheet, MC role at demo.

Timeline & Sync Points

Pre-hack (Thursday + Friday evenings): P0 prompts, all four tracks asynchronous.
Saturday 12:00–16:00 (4 hours): parallel build, sync points lock interfaces between tracks.

Time	T1 — Voice agent	T2 — Fleet pipeline	T3 — Dashboard	T4 — Identity + pitch
Pre-hack	P0.1 voice hello-world.	P0.2 pick 3 episodes; verify dataset format.	P0.3 Next.js scaffold + shadcn.	P0.4 generate 3 key-pairs, airdrop devnet.
12:00–12:30	Setup verify.	Setup verify.	Setup verify.	Setup verify.
12:30–13:00	Sync 1: API contracts. T1+T2 lock tool → fleet endpoint shapes. T2+T3 lock WS schemas. Write docs/api.			
13:00–14:00	P1: Voice loop (chat-only, no tools).	P2: VideoSource + 1 bridge running on real episode.	P3: Factory map view (3 tiles, click stub).	P4: Register 3 robots on devnet, mint metadata.
13:30	Sync 2 (light, 5 min): T2 publishes fleet endpoint URLs. T3 hardcodes them.			
14:00–15:00	P5: Five tools, multi-robot.	P6: 3 bridges in parallel + anomaly injector.	P7: Per-robot view with video + Auxin panels.	P8: Identity tile (handoff to T3) + pitch v1.
14:30	Sync 3: integration. Aura connects to live fleet. Dashboard connects to live fleet. Identity tile mount			
15:00–15:30	P9: Wire voice + tools + confirmation flow.	Support T1 + T3, fix what breaks.	P10: Aura transcript panel + selected-robot WS broadcast.	P11: Run-through 1 as MC.
15:30–15:45	Sync 4: dress rehearsal. T4 leads. Hotfix window after.			
15:45–16:00	Hard freeze. Plug into projector. Audio + mic check. Pre-load demo bag.			

Triage rules: if T2 falls behind on multi-bridge, drop to 2 robots. If T3 falls behind on per-robot view, ship the factory map only and demo by hovering. If T1 falls behind on the confirmation flow, ship reactive-only. Track 4 never falls behind — their work is parallelisable and their MC role is non-negotiable.

1 Phase 0: Pre-hack (Thursday & Friday Evenings)

P0.1: Voice Hello-World (T1, ~2h)

Tool: Cursor IDE **Model:** Claude Sonnet 4.6

Context: I am preparing for a 4-hour hackathon where I will build Aura, a voice-native operations agent for robot fleets. The single highest-risk technical item is the voice loop. Validate it tonight in throwaway code so Saturday is porting working code, never debugging APIs.

Task: Build a minimal end-to-end voice loop in /tmp/aura-hello/. Throwaway code, no tests, no polish.

Requirements:

1. Python 3.11, uv-managed venv. Single file main.py.
2. Press SPACE to start recording from default mic; SPACE again to stop. Record to 16kHz mono WAV in memory using sounddevice.
3. POST WAV to OpenAI Whisper (audio.transcriptions.create, model=whisper-1). Print transcript.
4. Send transcript to GPT-4o (chat.completions.create, model=gpt-4o, no system prompt, no tools). Stream reply text to stdout as it arrives.
5. As text streams, accumulate sentence-sized chunks (split on . ! ? + space). For each chunk, call ElevenLabs streaming TTS (voice=adam) and play through default speakers using sounddevice. Pipeline so user hears start of reply before generation completes.
6. Print round-trip wall-clock at end: SPACE-up to first audio sample played, SPACE-up to last sample played.
7. Target: first-audio under 2.5s. Last-audio under 5s for 2-sentence reply.

Constraints:

- API keys from env: OPENAI_API_KEY, ELEVENLABS_API_KEY. Crash loudly if missing.
- No frameworks. Just openai SDK, elevenlabs SDK, sounddevice, numpy.
- If PortAudio is fighting you on macOS, fall back to writing files + afplay via subprocess. Goal is a working loop, not elegance.

Output: code, then 5-line README on how to run.

Validation: talk to it for 30 seconds. If round-trip > 3s, profile and fix tonight, not Saturday.

P0.2: VLA Dataset Preparation (T2, ~1h)

Tool: Cursor IDE **Model:** Claude Sonnet 4.6

Context: I have a VLA dataset of real Franka Emika Panda episodes with this structure per session:

```
session_root/
  cameras/
    third_person_d405/
      rgb.mp4
      frames.jsonl      # one row per video frame: timestamp, frame_index
      metadata.json     # camera intrinsics, fps, resolution
    wrist_d405/
      rgb.mp4
      frames.jsonl
      metadata.json
  episode_events.jsonl  # high-level events: pick_started, pick_completed, anomaly, etc.
  robot.jsonl          # per-tick robot state: timestamp, joint_positions[7], joint_velocities[7], joint_torques[7], gripper_state, ee_pose
  session_metadata.json # episode-level metadata: task_description, duration_s, success_flag
```

Task: I need to (a) confirm three suitable episodes for the demo, (b) copy them into a fixed location, (c) write a one-page schema reference my teammates will use Saturday.

Steps:

1. Look at my dataset (path I will paste). Print a summary of each session's session_metadata.json: task_description, duration_s, success_flag, number of episode_events, joint count, fps of each camera.
2. Pick three episodes for the demo:
 - robot_01: routine successful pick-and-place, ~30s, no anomaly events.
 - robot_02: an episode with a clear anomaly event in episode_events.jsonl (gripper_slip, collision, etc).
 - robot_03: an idle or short episode ~15s.If my dataset doesn't contain a perfect fit, pick the closest and we'll fake the anomaly via T2's injector at runtime.
3. Copy the three sessions into aura-fleet/fixtures/episodes/{robot_01,robot_02,robot_03}/ preserving the directory structure.
4. Write aura-fleet/fixtures/episodes/SCHEMA.md (~80 lines):
 - Exact JSON schema of robot.jsonl (one example row, every field typed).
 - Exact JSON schema of frames.jsonl per camera.
 - Exact JSON schema of episode_events.jsonl (list of event types, payload shape per type).
 - Notes on timestamp alignment between robot.jsonl and frames.jsonl (same epoch? offset? frame-aligned?).
 - Notes on the joint convention (Franka 7-DOF, joint order, units for positions/velocities/torques).
 - Three sample queries: give me the joint state at t=2.5s, give me the third-person frame at t=2.5s, list events in window [t0, t1].

This document is the contract between T2's VideoSource and everything downstream. Get it right tonight.

P0.3: Dashboard Scaffold (T3, ~1h)

Tool: Cursor IDE **Model:** Claude Sonnet 4.6

Context: Saturday I have 4 hours to build a factory-floor dashboard for Aura. I want zero setup time on the day. Tonight, scaffold the project so Saturday I am only writing components, never fighting toolchains.

Task: Create aura-dashboard/ with Next.js 14 + TypeScript strict + Tailwind + shadcn/ui. Three placeholder pages.

Requirements:

1. `pnpm create next-app@latest aura-dashboard --typescript --tailwind --app --use-pnpm`. Eslint on, src dir on.
2. Install shadcn/ui, init, add components: card, badge, button, dialog, separator, scroll-area, skeleton.
3. Create three routes:
 - / (factory floor view --- placeholder grid of 3 cards)
 - /robot/[id] (per-robot view --- placeholder with three sections: video, telemetry, identity)
 - /aura (Aura transcript panel --- placeholder list)
4. Header bar: AURA wordmark left (custom font, condensed sans, uppercase), connection status dot right, current selected robot ID centre.
5. Tailwind theme: dark mode default. Define brand colors in tailwind.config.ts: primary (electric blue #2563eb), warning (amber #f59e0b), danger (red #ef4444), success (emerald #10b981), surface tones for dark cards.
6. Mock data file src/lib/mockFleet.ts: array of 3 robots with id, name, status, grade, last_anomaly. Use this until T2's fleet endpoints are live.
7. Confirm `pnpm build` succeeds with zero errors and zero ESLint warnings.
8. Commit and push.

Style anchors:

- Inspiration: shadcn/ui defaults + a hint of industrial control panel (tight type, monospace numerics, subtle grid background).
- DO NOT use gradients or glow effects. Restrained, professional.
- Numerics in JetBrains Mono or similar. Headings in Inter.

Output: confirm pnpm build is clean, paste the file tree.

P0.4: Three Robot Wallets (T4, ~30min)

Tool: terminal + solana CLI **Model:** —

Goal: three pre-funded Solana devnet keypairs ready for Saturday, plus three robot metadata JSONs.

Steps:

1. Generate three keypairs:
`solana-keygen new --outfile ~/.config/aura/robot_01.json --no-passphrase`
`solana-keygen new --outfile ~/.config/aura/robot_02.json --no-passphrase`
`solana-keygen new --outfile ~/.config/aura/robot_03.json --no-passphrase`
2. Airdrop on devnet (each):
`solana airdrop 2 <pubkey> --url https://api.devnet.solana.com`
Confirm balance ≥ 1.5 SOL each. Devnet airdrop has rate limits; if it fails, retry tomorrow.
3. Write three metadata JSONs to `aura-dashboard/public/robot_metadata/{robot_01,robot_02,robot_03}.json` with this schema:

```
{
  name: Aura-Panda-01,
  manufacturer: Franka Emika,
  model: Panda 7-DOF,
  serial: FE-PANDA-2024-0001,
  deployment_date: 2026-04-01,
  operator: Aura Demo Fleet,
  wallet_pubkey: <pubkey from step 1>,
  episode_hash: <sha256 of the episode directory tarball>,
  image_url: /robot_images/panda_01.png
}
```
4. Drop three Franka product images into `aura-dashboard/public/robot_images/`. Do not use Boston Dynamics or other licensed images. Use Franka Emika's official press images (publicly available) or generate minimalist line-art stand-ins.

Pre-hack only. The on-chain registration via Auxin's `initialize_agent` happens in P4 Saturday, when we have the agent PDAs configured.

2 Phase 1: Saturday 12:00–12:30 — Setup Verification (all tracks)

P1.0: Verification Checklist (joint, 30min)

Tool: terminal + browser **Model:** —

At 12:00 sharp, all four people run their tracks' verification independently. No code is written until all four tracks PASS.

T1 --- Aura voice agent verification:

- /tmp/aura-hello (P0.1) runs end-to-end with current API keys.
- Round-trip latency under 3s.
- ELEVENLABS_API_KEY upgraded to Scale tier (sponsor credit redeemed in first 5 min).

T2 --- Fleet pipeline verification:

- aura-fleet/fixtures/episodes/{robot_01,robot_02,robot_03}/ exist and contain rgb.mp4 + frames.jsonl + robot.jsonl + episode_events.jsonl.
- SCHEMA.md is readable and accurate.
- Auxin's bridge runs locally on this laptop with AUXIN_SOURCE=mock against devnet (sanity check before swapping in VideoSource).

T3 --- Dashboard verification:

- aura-dashboard/ pnpm build is clean.
- pnpm dev runs at localhost:3000, three placeholder routes render.
- Robot images and metadata JSONs in public/ are accessible at expected URLs.

T4 --- Identity + pitch verification:

- Three keypairs in ~/.config/aura/ with ≥ 1.5 SOL devnet balance each.
- Three metadata JSONs in dashboard public/.
- A printed copy of the prelim pitch outline (rough, will be finalised P8).

If any track FAILs at 12:30, that track's owner stops and fixes before joining Sync 1. The cost of debugging at 13:00 is much higher than at 12:30.

3 Phase 2: Saturday 12:30–13:00 — Sync 1: API Contracts

P1.1: Lock Interfaces Between Tracks (T1 + T2 + T3, 30min)

Tool: shared markdown editor **Model:** Claude Sonnet 4.6 (drafting only; humans negotiate)

Context: Four tracks are about to build in parallel. The boundaries between them are the project's biggest risk. We lock them now so nobody is debugging an integration mismatch at 14:30.

Task: Three people (T1, T2, T3) write aura/docs/api_contract.md together. T4 reads and signs off.

Required sections:

1. Fleet HTTP endpoints (T2 commits to providing):

- GET /fleet/status returns: { robots: [{ id, name, wallet_pubkey, status: healthy | anomaly | idle | paused, grade: A..F, runway_hours, latest_event_ts, video_url, telemetry_ws_url, compliance_ws_url }] }
- GET /robot/{id}/status returns single robot's status block.
- GET /robot/{id}/recent_events?n=N returns last N compliance events for that robot, with tx_signatures.
- POST /robot/{id}/inject_anomaly triggers a fake HIGH-severity event (used by T4 mid-demo).
- POST /robot/{id}/pause sets a pause flag the VideoSource reads (the VideoSource freezes on the current frame).
- GET /robot/{id}/identity returns metadata JSON (proxied from T4's chain registration).
- All on port 8780 (the fleet aggregator port; individual bridges are 8767+i).

2. WebSocket schemas (T2 commits to providing, T1 + T3 consume):
 - ws://localhost:8770/aura --- Aura transcript broadcaster (T1 publishes, T3 subscribes). Message format from playbook P10.
 - ws://localhost:8780/fleet --- fleet event multiplex (T2 publishes, T3 subscribes). Messages : { robot_id, type: telemetry | compliance | payment | status_change, payload: ... }.
 - ws://localhost:8780/selected --- selected-robot channel (T3 publishes when user clicks a tile, T1 subscribes for ambient context).
3. Tool argument shapes (T1 commits to using):
 - All five Aura tools accept an OPTIONAL robot_id arg. If absent, the tool uses Aura's currently-selected robot from the WS context.
 - Tools: get_fleet_status (no robot_id, returns all), get_recent_compliance_events (robot_id required), investigate_anomaly (robot_id + event_id required, costs SOL), generate_incident_report (optional robot_id; if absent, fleet-wide report), pause_fleet (optional robot_id; if absent, all robots).
4. Robot identity payload (T4 commits to providing, T3 displays):
 - Path: /robot_metadata/{robot_id}.json (served as static asset by dashboard).
 - On-chain registration: AgentInitializedEvent emitted by Auxin program with hardware_pubkey == robot wallet. T4 stores tx_signatures of the three init transactions in metadata.json's registration_tx field.
5. Failure modes (joint commitment):
 - If a bridge process dies, fleet aggregator returns status: offline for that robot. Dashboard greys out the tile.
 - If ElevenLabs rate-limits, voice loop falls back to OpenAI tts-1.
 - If devnet RPC is slow, all writes are best-effort with 5s timeouts; demo never blocks on chain confirmation.

Acceptance: every box on this contract is implementable in the 13:00--14:30 window. If anything is risky, scope it down or move it to stretch before signing off.

T4 reads, sanity-checks, and approves at 13:00. Then everyone goes to their track.

4 Phase 3: Saturday 13:00–14:00 — First Build Block

P1: Aura Voice Loop (T1, 60min)

Tool: Cursor IDE **Model:** Claude Sonnet 4.6

Context: Port the validated voice hello-world (P0.1) into `aura-agent/src/aura/voice.py` with structure for the full agent. Chat-only loop here; tools come in P5; wiring in P9.

Task: Implement `aura-agent/src/aura/voice.py`.

Requirements:

1. class `VoiceLoop` with constructor(`reasoning_client`, `voice_id`, `system_prompt`). `reasoning_client` is injected: `async def reason(messages: list[dict]) -> AsyncIterator[str]`.
2. `async def turn() -> tuple[str, str]`: records mic until silence detected OR SPACE pressed (support both), transcribes via Whisper, calls `reasoning_client`, streams TTS to speakers, returns (`user_transcript`, `assistant_reply`).
3. Sentence-chunked TTS: as text streams from `reasoning_client`, accumulate until `[.!?] + space`, send to ElevenLabs streaming, pipeline playback.
4. `async def run()`: loops `turn()` until `KeyboardInterrupt`. Maintains conversation history. After each turn, broadcasts the (`user`, `assistant`) pair to `ws://localhost:8770/aura` per the WS contract.
5. System prompt at top of file:

```
SYSTEM_PROMPT = You are Aura, the voice operations agent for an autonomous robot fleet.
The fleet has three Franka Emika Panda arms, each with its own Solana wallet.
You speak concisely. Two-sentence answers by default.
You never invent telemetry --- if you don't have data, say so.
You confirm aloud before any action that costs money.
Your tone is calm, professional, slightly warm. You are not a chatbot; you are a fleet
operator's colleague.
```

6. Latency target: first-audio-played under 2.5s from end-of-recording. Print measured latency in dim grey after each turn.
7. No tool use here. `voice.py` is a clean voice $\leftrightarrow$ text adapter.
8. WS broadcasting: also start a tiny aiohttp WS server on `:8770` so the dashboard can subscribe. Buffer the last 20 turns for late-joining clients.

Smoke test: `tests/test_voice_smoke.py` mocks all external APIs and asserts `turn()` returns a tuple with non-empty strings under 100ms.

End-of-block validation: have a 30-second conversation. Two-way must feel natural, latency under 3s. If not, debug now before P5.

P2: VideoSource + One Bridge Running (T2, 60min)

Tool: Cursor IDE **Model:** Claude Opus 4.7 (timestamp alignment is fiddly — worth Opus's reasoning)

Context: Implementing a `TelemetrySource` for Auxin that replays real Franka VLA episodes as live telemetry. The schema reference is `fixtures/episodes/SCHEMA.md` from P0.2. The `TelemetrySource ABC` is at `Auxin_Automata/sdk/src/auxin_sdk/sources/base.py`.

Task: Implement `Auxin_Automata/sdk/src/auxin_sdk/sources/video.py` and verify it runs end-to-end with one bridge instance.

Requirements:

1. class `VideoSource(TelemetrySource)`. Constructor:
 - `episode_dir`: Path (e.g. `fixtures/episodes/robot_01/`)
 - `rate_hz`: int = 10 (target Auxin telemetry rate)
 - `loop`: bool = True (restart episode on completion --- demo runs 30+ minutes)

- speed: float = 1.0 (1.0 = realtime; tunable for debug)

2. `async def stream() -> AsyncIterator[TelemetryFrame]:`
 - On start, load `robot.jsonl` into memory as a list of state dicts (these are small --- at 100 Hz, 60s = 6000 rows, trivial).
 - Compute the original recording rate from timestamps. Resample to `rate_hz`: take the row whose timestamp is closest to (`current_episode_time`, in source frame) for each output tick.
 - For each output frame, build a `TelemetryFrame`:
 - `timestamp` = `datetime.now(UTC)`
 - `joint_positions` = `robot.jsonl` row's `joint_positions[0:6]` (drop joint 7 OR pad to 6 --- match Auxin's existing schema; flag in code with `TODO` if joint count differs)
 - `joint_velocities`, `joint_torques`: same.
 - `end_effector_pose`: from `ee_pose`.
 - `anomaly_flags`: see step 3.
 - Sleep until next tick (use `anyio.sleep`, not `time.sleep`).
 - On loop: when episode ends, log a `loop_restart` `structlog` event and start over.
3. Anomaly mapping --- the only non-trivial part:
 - Maintain a pointer into `episode_events.jsonl`. As the episode timestamp advances, check if any event's timestamp falls in the current tick.
 - For each event whose type is in `{gripper_slip, collision, joint_torque_spike, anomaly, error}`, append the type as a string to the current frame's `anomaly_flags`.
 - All other event types (`pick_started`, `pick_completed`, etc.) are ignored for `anomaly_flags`.
 - Document the mapping in a docstring; T4's pitch refers to it.
4. Pause flag: read `/tmp/aura_pause_{robot_id}` on each tick (`robot_id` from constructor). If present, freeze on the current frame (yield the same frame on every tick) until the file is removed.
5. Update Auxin's `run_bridge.py` to support `AUXIN_SOURCE=video` plus `AUXIN_VIDEO_EPISODE_DIR` env var. Do this with the SMALLEST possible diff --- a 5-line factory addition. If you find yourself rewriting `bridge.py`, stop and reconsider.
6. Smoke test: `aura-fleet/scripts/smoke_video_source.py` instantiates `VideoSource` on `robot_01`, streams 30 frames, asserts no exceptions, prints a sample frame.
7. End-of-block validation: ONE bridge runs against devnet with `VideoSource(robot_01)`. The Auxin dashboard at `localhost:3000` (using existing Auxin code) shows live telemetry from the real Franka episode. Compliance events fire when episode_events anomalies appear. Payments fire on each oracle approval (the Gemini oracle still runs --- it just gets real Franka workspace images now, IF you can wire the `rgb.mp4` frames to it; otherwise pass a placeholder image and the oracle uses the heuristic fallback).

Critical: do not over-engineer. We need ONE bridge running on real data by 14:00. Multi-bridge is P6.

P3: Factory Floor View (T3, 60min)

Tool: Cursor IDE **Model:** Claude Sonnet 4.6

Context: Building the factory floor landing page. Three robot tiles arranged on a 2D top-down layout, each clickable into a per-robot detail view. This is the dashboard's hero shot --- judges see this first.

Task: Implement `aura-dashboard/src/app/page.tsx` and supporting components.

Requirements:

1. `src/app/page.tsx` renders a `FactoryFloor` component.
2. `src/components/FactoryFloor.tsx`:

- Background: subtle dark grid (CSS background-image with linear-gradient grid pattern, 40px squares, opacity 0.05). Aspect-ratio container, 16:9, max-width 1400px, centred.
 - Three robot tiles positioned on the grid (absolute positioning with predetermined coordinates): robot_01 top-left quadrant, robot_02 centre, robot_03 bottom-right.
 - Static decorative elements: a loading dock label top edge, a QC station label right edge, a packing line label bottom edge. Sets the factory mental model without any actual visualisation work.
3. src/components/RobotTile.tsx:
 - Card sized 240x180px.
 - Header: robot name in mono uppercase (AURA-PANDA-01), status badge top-right (colour-coded: green healthy, amber idle, red anomaly).
 - Body: grade letter LARGE (text-4xl font-bold), runway_hours below in small mono (42h runway), latest_event_ts as relative time (2m ago).
 - Footer: wallet pubkey truncated (first 4 + last 4 of base58), tiny external-link icon to Solana Explorer (devnet cluster).
 - Hover: subtle lift + glow in the status colour.
 - Click: router.push(`/robot/\${id}`).
 - Polling: fetch GET /fleet/status every 2s (use SWR or react-query). Until T2's fleet endpoint is live, fall back to mockFleet.ts.
 4. Header bar (src/components/Header.tsx, mounted in app/layout.tsx):
 - Left: AURA wordmark (Inter, bold, uppercase, tracking-widest).
 - Centre: current selected robot ID (empty on factory floor view).
 - Right: connection status dot (green if /fleet/status is reachable, red otherwise). Mono timestamp of last successful poll.
 5. Status badge logic (extract to src/lib/status.ts):
 - healthy -> green (\$success), idle -> amber (\$warning), anomaly -> red (\$danger), paused -> grey, offline -> dark grey with strikethrough text.
 6. Style commitment: NO gradients, NO shadows beyond shadcn defaults, NO icons beyond lucide-react. Industrial control panel discipline. Numerics in JetBrains Mono.
 7. End-of-block validation: localhost:3000 shows three tiles with mock data. Click a tile, route changes to /robot/robot_01, placeholder page renders. Header shows connection status. pnpm build clean.

P4: On-chain Robot Registration (T4, 60min)

Tool: Cursor IDE + terminal **Model:** Claude Sonnet 4.6

Context: Each of the three robot wallets needs to be registered on Auxin's deployed Solana program via the existing initialize_agent instruction, then the resulting tx_signature recorded into the metadata JSONs so the dashboard can link to each robot's identity proof on Solana Explorer.

Task: Register three robots and update metadata.

Requirements:

1. Write aura-fleet/scripts/register_robots.py:
 - For each of robot_01, robot_02, robot_03:
 - Load keypair from ~/.config/aura/{robot_id}.json.
 - Call Auxin's initialize_agent instruction with:
 - owner = a single shared operator keypair (use the existing owner.json from Auxin demo)
 - hardware_pubkey = robot wallet pubkey
 - compute_budget_lamports = 100_000_000 (0.1 SOL)
 - Whitelist the standard provider PDA from Auxin's existing scripts.
 - Print resulting agent_pda + tx_signature.
 - Idempotent: if the agent PDA already exists, skip with a log.

2. Update each `aura-dashboard/public/robot_metadata/{robot_id}.json` with:
 `registration_tx`: <tx signature from step 1>,
 `agent_pda`: <derived PDA>,
 `explorer_url`: `https://explorer.solana.com/address/<agent_pda>?cluster=devnet`
3. Verify each registration on Solana Explorer manually (open three tabs).
4. Write the `IdentityTile` component (handed off to T3 to mount in `/robot/[id]` at P7):
 `src/components/IdentityTile.tsx`
 - Fetch `/robot_metadata/{id}.json`.
 - Render: robot image, name + manufacturer + serial, deployment date, wallet pubkey (clickable to Explorer), agent PDA (clickable to Explorer), registration tx (clickable to Explorer), episode hash (mono, full string).
 - At the bottom: a small green Verified on chain badge with the Solana logo.
5. Pitch v1 outline (`docs/pitch_v1.md`, rough draft, will be finalised in P8):
 - Three sentences for the hook.
 - Three-act demo skeleton: status check (act 1), anomaly investigation with paid call + on-chain receipt (act 2), report + identity proof (act 3).
 - One sentence for the close.

End-of-block: three robots verifiably registered on devnet, `IdentityTile` component ready to mount.

P5: Five Multi-Robot Tools (T1, 60min)

Tool: Cursor IDE **Model:** Claude Sonnet 4.6

Context: Implementing Aura's tool layer per the API contract. Each tool wraps fleet endpoints from T2. All tools accept an optional robot_id; if absent, fall back to Aura's currently-selected-robot ambient context.

Task: Implement aura-agent/src/aura/tools/*.py and the registry.

Shared infrastructure (aura/tools/fleet_client.py):

1. class FleetClient: thin httpx wrapper.
 - async def get_fleet_status() -> FleetStatus
 - async def get_robot_status(robot_id) -> RobotStatus
 - async def get_recent_events(robot_id, n) -> list[ComplianceEvent]
 - async def trigger_inference_payment(robot_id, lamports, reason) -> str (calls Auxin's stream_compute_payment via the bridge for that robot)
 - async def trigger_compliance_log(robot_id, payload) -> str (calls log_compliance_event)
 - async def pause_robot(robot_id) -> None
 - async def get_identity(robot_id) -> dict
2. class SelectedRobotContext: subscribes to ws://localhost:8780/selected, holds latest selected robot_id. Tools that omit robot_id read from this.
3. ToolResult model: tool_name, success, robot_id, result, cost_lamports, tx_signature, reasoning (spoken summary).
4. Registry pattern in tools/__init__.py: TOOLS dict mapping name -> ToolSpec(schema_for_llm, handler).

Tool 1 --- get_fleet_status:

Args: none.
Returns aggregated fleet status. Cost: 0.
reasoning: All three robots online. Robot 1 grade A, healthy. Robot 2 grade C, one open anomaly. Robot 3 idle.

Tool 2 --- get_recent_events:

Args: { robot_id: str (optional), n: int default 5 }.
Returns last N compliance events for the specified or selected robot.
Cost: 0.
reasoning: human-readable summary of events with relative timestamps.

Tool 3 --- investigate_anomaly:

Args: { robot_id: str (optional), event_id: str (a tx_signature) }.
Behavior: pull telemetry window around event from fleet endpoint, send to GPT-4o with a structured diagnostic prompt, fire trigger_inference_payment(robot_id, 800000, anomaly diagnosis).
Cost: 800000 lamports (0.0008 SOL --- under Auxin's 0.001 cap).
reasoning: spoken diagnosis with proximate cause and recommended action.
THE diagnostic prompt:
You are an industrial robotics diagnostician. Given this telemetry window from a Franka Emika Panda arm around a flagged compliance event, identify the proximate cause and recommend an action. Reply as JSON: { proximate_cause, severity_assessment, recommended_action, confidence_pct }. Be concise; this will be read aloud to a human operator.

Tool 4 --- generate_incident_report:

Args: { robot_id: str (optional), from_iso: str, to_iso: str }.
Behavior: trigger Auxin's existing weasyprint pipeline scoped to one robot or fleet-wide.
Returns dashboard URL of the served PDF.
Cost: 0.

Tool 5 --- pause_robot:
Args: { robot_id: str (optional) }.
Behavior: POST /robot/{id}/pause to fleet endpoint. The VideoSource freezes on the current frame.
Cost: 0.
reasoning: Robot {id} paused. Resume with 'unpause robot {id}' or click resume in the dashboard.

Critical:

- Every tool MUST log a compliance event with tool_name + args. The on-chain log is ground truth of what Aura did.
- Tool 3 is the ONLY tool that costs SOL. Cost hardcoded; do NOT parametrise over the LLM.
- 5-second timeout per tool. On timeout: ToolResult(success=False, reasoning=That timed out, want me to retry or skip?).

Smoke tests in scripts/smoke_tools.py: call each tool with hardcoded args for robot_01, print ToolResult. Verify each produces an on-chain compliance event.

P6: Three Bridges in Parallel + Anomaly Injector (T2, 60min)

Tool: Cursor IDE **Model:** Claude Sonnet 4.6

Context: One bridge runs on real video data after P2. Now we run three in parallel and add a fleet aggregator that exposes the API contract from Sync 1.

Task: Implement aura-fleet/scripts/run_fleet.py and aura-fleet/src/aura_fleet/aggregator.py.

Requirements:

1. aura-fleet/fleet.yaml (fleet config):
robots:
 - id: robot_01
name: Aura-Panda-01
keypair: ~/.config/aura/robot_01.json
episode: fixtures/episodes/robot_01
bridge_http_port: 8767
bridge_ws_port: 8766
 - id: robot_02
(same shape, ports 8768/8771)
 - id: robot_03
(same shape, ports 8769/8772)
2. scripts/run_fleet.py:
 - Reads fleet.yaml.
 - For each robot, spawns an Auxin bridge subprocess via asyncio.create_subprocess_exec with env vars set:
AUXIN_SOURCE=video
AUXIN_VIDEO_EPISODE_DIR=<episode>
HW_KEYPAIR_PATH=<keypair>
BRIDGE_HEALTHZ_PORT=<port>
BRIDGE_WS_PORT=<port>
AUXIN_PRIVACY=direct
PROMETHEUS_ENABLED=false (kill metrics; saves RAM)
 - Spawns the aggregator on port 8780.
 - Streams subprocess stdout/stderr with a [robot_01], [robot_02], [robot_03] prefix per line. Coloured.
 - Graceful shutdown on SIGINT --- terminates all children.
 - Health: every 5s, GET each bridge's /healthz, log status. If a bridge dies, log error and continue (other robots stay up).

3. `src/aura_fleet/aggregator.py`: aiohttp server on :8780.
Endpoints (matching Sync 1 contract):
 - GET `/fleet/status`: queries each bridge's `/healthz` in parallel, joins with metadata, returns the contracted shape.
 - GET `/robot/{id}/status`: single bridge status block.
 - GET `/robot/{id}/recent_events?n=N`: queries Solana directly for ComplianceEvents emitted by that robot's agent PDA. (NOT proxied through bridge --- bridges don't expose this; we go to chain. Use `solana-py` and `parseProgramLogs`.)
 - POST `/robot/{id}/inject_anomaly`: writes a JSON message to the bridge's anomaly-inject WS endpoint OR a sentinel file; the VideoSource (extended below) reads it and adds `injected_anomaly` to the next frame's `anomaly_flags`.
 - POST `/robot/{id}/pause`: writes `/tmp/aura_pause_{robot_id}`; the VideoSource reads it and freezes.
 - GET `/robot/{id}/identity`: reads `aura-dashboard/public/robot_metadata/{robot_id}.json`.
 - WS `/fleet`: multiplexes telemetry/compliance/payment events from all three bridge WS streams. Each message tagged with `robot_id`.
4. Extend `video.py` with the inject mechanism: a sentinel file `/tmp/aura_inject_{robot_id}` that, when present, adds `injected_anomaly` to the next frame's `anomaly_flags` and deletes the file. This is a 10-line addition.
5. End-of-block validation:
 - `run_fleet.py` starts cleanly.
 - GET `http://localhost:8780/fleet/status` returns three robots, all healthy.
 - WS to `ws://localhost:8780/fleet` streams telemetry from all three.
 - POST to `/robot/robot_02/inject_anomaly` causes a compliance event to fire on chain for `robot_02` within 2s.
 - All on a 16GB laptop without OOM. (If RAM is tight: drop privacy providers, drop Prometheus, set `rate_hz=5` not 10.)

Critical: do NOT modify `Auxin bridge.py` beyond what's needed for `AUXIN_SOURCE=video`. The aggregator is in `aura-fleet`, not in `Auxin`.

P7: Per-Robot Detail View (T3, 60min)

Tool: Cursor IDE **Model:** Claude Sonnet 4.6

Context: When the user clicks a tile on the factory floor, they land at `/robot/[id]`. This is where they spend most of the demo. Two video panes (third-person + wrist), live telemetry, compliance log, identity proof.

Task: Implement `aura-dashboard/src/app/robot/[id]/page.tsx` and supporting components.

Layout (mobile-first not needed; design for projector at 1920x1080):

- Top header: robot name large, status badge, back to factory floor link. Left of the back link: the AURA wordmark (clickable home).
- Main grid (3 columns):
 - Left column (50% width): video stack.
 - Top: third-person camera (16:9, ~600x340).
 - Bottom: wrist camera (4:3, ~600x450).
 - Right column (50% width):
 - LiveTelemetry: a compact panel with current joint positions (7 numerics in mono), gripper state, EE pose. Updates from WS.
 - ComplianceLog: scrolling list, last 10 events. Each row: timestamp, severity badge, reason_code, hash (truncated, click to copy), tx link to Explorer. Updates live.
 - PaymentTicker: recent payments, each row: timestamp, lamports, provider (truncated), tx link. Live.
 - IdentityTile: from T4's component.

Requirements:

1. `src/app/robot/[id]/page.tsx`:
 - Reads `id` from route params. Fetches `/api/robot/{id}/status` on mount AND subscribes to WS `ws://localhost:8780/fleet`, filters for matching `robot_id`.
 - On WS disconnect: shows a banner `Disconnected from fleet --- attempting to reconnect...` with a count.
2. `src/components/VideoFeed.tsx`:
 - Props: `src` (video URL), `title`, `looped` (bool).
 - Renders: an `<video>` element with `autoplay`, `muted`, `loop`, `playsInline`. controls hidden. Title bar above with the camera name.
 - For the demo: video `src` is `/videos/{robot_id}/third_person.mp4` served as a static asset. Copy the `rgb.mp4` files from `fixtures/episodes` into `aura-dashboard/public/videos/{id}/{camera}.mp4` once at build time.
 - Optional later: sync video playback to the bridge's reported episode time. For now, just loop independently --- judges won't notice 1-2s drift.
3. `src/components/LiveTelemetry.tsx`, `ComplianceLog.tsx`, `PaymentTicker.tsx`:
 - Each subscribes to the filtered WS stream.
 - `LiveTelemetry`: 7 joint values shown as a mono 7-row vertical list with 3-decimal precision, plus a tiny inline sparkline of the last 50 samples for each (use `recharts MiniBar` or a custom SVG `<polyline>`; either works).
 - `ComplianceLog`: scroll-area, max-height 320px, newest at top, fade-in animation on new entry.
 - `PaymentTicker`: same shape, smaller font.
4. Identity tile mounting: import T4's `IdentityTile` component, place at bottom of right column.
5. Selected-robot broadcast: on mount of `/robot/[id]`, send a WS message to `ws://localhost:8780/selected` with `{ selected_robot_id: id }`. On unmount: send `{ selected_robot_id: null }`. T1's agent listens on this channel for ambient context.
6. End-of-block validation: navigate to `/robot/robot_02`. Both videos play. Telemetry updates live. A compliance event fired by P6's anomaly injector appears in the `ComplianceLog` within 2s of injection. Identity tile renders with the Explorer link working.

P8: Pitch v2 + Run Sheet (T4, 60min)

Tool: markdown editor **Model:** Claude Opus 4.7

Context: Demos run 16:00. Three minutes per team. Judges include David Helgason (Transition VC, ex-Unity), NEA partners, sponsor reps. Audience is half engineers, half investors. The pitch lands in the first 30 seconds or it doesn't land at all.

Task: Write `docs/pitch.md` and `docs/run_sheet.md`. Refines `pitch_v1.md` from P4.

`docs/pitch.md` (~250 words, three sections):

1. THE 30-SECOND HOOK. One paragraph. Open with a concrete, image-rich problem (A robot arm crushes a pallet at 3am. Six lawyers spend three weeks arguing about what data to trust. The fleet manager has 200 other robots to watch and is reading PDFs.). Frame the gap. Drop a one-line product description: Aura is the voice interface for a robot fleet where every machine has its own wallet and every action has an immutable receipt. Include one phrase the partners will repeat to a colleague --- robots that pay for their own intelligence or similar.
2. THE 90-SECOND DEMO. Three-act script with OPERATOR (Track 4 holding the mic) and AURA, on-screen events called out as italics:

ACT 1 --- the factory.
OPERATOR: Aura, status check on the fleet.

(Factory floor view visible: three tiles, three colour-coded badges)
AURA: All three robots online. Robot 1 grade A, healthy. Robot 2 grade C, one open anomaly. Robot 3 idle.
OPERATOR: *(clicks Robot 2)* Tell me about Robot 2.
(View transitions to /robot/robot_02. Two video feeds, telemetry streaming)
AURA: Robot 2 logged a torque spike on joint 3 four minutes ago. The hash is on chain. Want me to investigate?

ACT 2 --- the paid investigation.
OPERATOR: Yes.
AURA: That call to GPT-4o will cost about half a cent of SOL. Proceed?
OPERATOR: Go.
(PaymentTicker: a payment fires. ComplianceLog: a new investigation event with tx link)
AURA: Diagnosis: gripper slip during a pick at the QC station. Recommended action: pause for inspection.

ACT 3 --- the receipt.
OPERATOR: Pause it. Generate the report. Show me Robot 2's identity.
(Video freezes on current frame. PDF appears. IdentityTile is in view)
AURA: Robot 2 paused. Report ready. This is its identity --- registered on chain on April 1st, 47 anomaly events in its lifetime, all signed by its own wallet. Every receipt traceable.

The script reads aloud in 90 seconds at calm pace. Memorise the phrasing for Aura's most quotable lines.

3. THE 30-SECOND CLOSE.

Every robot fleet is going to need this substrate. The EU AI Act lands next year. Insurance and warranty for autonomous machines is a \$40 billion problem with no current answer. Aura is the voice operator for a fleet where every action has a receipt and every robot has an identity. We are building this. We'd love 15 minutes with David --- and with NEA --- to show what's next.

docs/run_sheet.md (~200 words):

- Roles: T4 holds mic + narrates clicks. T3 operates dashboard. T1 sits at agent terminal, can intervene if recognition fails. T2 sits at fleet terminal, has anomaly-injector ready.
- Cue sheet:
 - 0:00 OPERATOR speaks first line.
 - 0:25 OPERATOR clicks Robot 2.
 - 0:50 T2 silently triggers POST /robot/robot_02/inject_anomaly so the open anomaly Aura mentions is real and on chain.
 - 1:30 OPERATOR says Pause it...
 - 2:30 close.
- 15:45 freeze checklist: HDMI plugged, audio output set, mic line-level checked, demo bag pre-loaded, dashboard windows arranged on /, browser zoomed for projector visibility, terminal font size up.
- Plan B per failure mode (one line each):
 - Whisper slow: T1 has hardcoded buttons status / investigate / pause.
 - ElevenLabs rate-limited: voice loop falls back to OpenAI tts-1.
 - Devnet RPC slow: skip waiting for tx confirmation; show submitting... optimistically.
 - Bridge dies mid-demo: show pre-recorded incident from demo bag; T4 narrates as earlier this morning.
 - Confirmation classifier misclassifies: T1 types y in the terminal as override.
 - Video doesn't play: refresh the per-robot page once.

Tone: confident, technical, no marketing fluff. Helgason is an engineer first.

P9: Voice + Tools + Confirmation Flow (T1, 30min)

Tool: Cursor IDE **Model:** Claude Opus 4.7

Context: P1 voice loop and P5 tools are both ready. This is the integration: voice + tools + the confirmation flow that is Aura's signature demo moment.

Task: Implement `aura-agent/src/aura/agent.py`. Update `scripts/run_aura.py`.

Requirements:

1. `class AuraAgent(VoiceLoop)`. Inherits voice scaffolding; overrides reasoning client with tool-using one.
2. Tool-using reasoning client:
 - Pass `TOOLS` dict (P5) as tools spec to OpenAI chat completions API with `tool_choice=auto`.
 - Loop: send conversation -> receive reply (may contain `tool_calls`) -> for each `tool_call`, dispatch to handler, append tool message with result -> repeat until model returns final text.
 - Stream final text through voice loop's TTS pipeline.
3. THE CONFIRMATION FLOW (the demo moment):
 - Before dispatching ANY tool where `TOOLS[name].cost_lamports > 0`, agent first speaks a confirmation prompt and waits for verbal yes/no.
 - When LLM emits `tool_call` for ``investigate_anomaly``:
 - a. Do NOT execute tool yet.
 - b. Generate confirmation: That call to GPT-4o for anomaly diagnosis will cost about half a cent of SOL. Proceed?
 - c. Stream confirmation through TTS.
 - d. Record next user turn (fresh mic capture, max 5s).
 - e. Transcribe + classify intent: regex match {yes, yeah, go, do it, proceed, confirm, sure} -> execute. Match {no, stop, cancel, wait, hold} -> abort with `tool_result {aborted: true}`, let LLM continue.
 - f. Ambiguous: one-shot Claude Haiku 4.5 (Does this utterance mean YES or NO? One word.) with 1s timeout. Default NO on timeout.
 - All confirmation prompts and decisions logged on chain as compliance events: `tool_name=confirmation_request / confirmation_resolved`.
4. Tool result narration:
 - After every `tool_call` returns, BEFORE next LLM turn, speak the `tool_result.reasoning` field. Makes Aura feel responsive on slow tools.
5. Selected-robot ambient context:
 - Subscribe to `ws://localhost:8780/selected`.
 - When the dashboard broadcasts a new `selected_robot_id`, inject a system message into the conversation: `[CONTEXT] Operator now focused on robot {id}`.
 - Tool calls without `robot_id` default to the selected one.
6. Dashboard broadcasting:
 - Every user turn, assistant turn, `tool_call`, `tool_result` broadcast to `ws://localhost:8770/aura` per Sync 1 contract.
7. Updated `SYSTEM_PROMPT`:

You are Aura, the voice operations agent for an autonomous robot fleet of three Franka Emika Panda arms.

Each robot has its own Solana wallet and pays for its own cloud inference.

You have five tools: `get_fleet_status`, `get_recent_events`, `investigate_anomaly`, `generate_incident_report`, `pause_robot`.

Use them to answer questions truthfully. NEVER fabricate telemetry.

When a tool costs money, you MUST confirm verbally before calling it; the system handles the confirmation flow --- just emit the `tool_call` normally.

The operator's currently-focused robot is provided in [CONTEXT] system messages; use it as the default robot_id.

Reply concisely; two sentences by default. Mention specific robot IDs when relevant.

8. scripts/run_aura.py:

- Loads .env, instantiates FleetClient, AuraAgent, the dashboard WS broadcaster, the selected -robot subscriber.
- Starts everything as anyio tasks. Runs until SIGINT.

End-of-block validation: full run-through of pitch.md Act 1 and Act 2, end-to-end, with real on-chain payment fired.

P10: Aura Transcript Panel (T3, 30min)

Tool: Cursor IDE **Model:** Claude Sonnet 4.6

Context: Persistent panel on the dashboard showing live voice exchange between operator and Aura, with tool calls expanded inline. Visible from both factory floor view AND per-robot view (it's persistent across navigation).

Task: Implement src/components/AuraConversation.tsx. Mount in app/layout.tsx as a persistent right-side panel (collapsible).

Requirements:

1. Subscribes to ws://localhost:8770/aura. Message schema (matches Sync 1 contract):

```
{ type: user_turn | assistant_turn | tool_call | tool_result,
  timestamp, content, tool_name?, tool_args?, tool_result?, cost_lamports?, tx_signature? }
```
2. Render: vertical scroll, newest at bottom (auto-scroll). Two-column inside the panel: operator messages left-aligned (icon: person), Aura right-aligned (icon: brand mark). Tool_call cards rendered inline between turns: collapsible, mono header showing tool name + args, expanded shows tool_result; if tx_signature present, a clickable link to Solana Explorer (devnet) with a copy-to-clipboard button.
3. Style:
 - Mounted as a 360px-wide right rail, collapsible to a 32px tab.
 - Background: same surface as cards.
 - User messages: light text. Aura messages: bright primary colour. Tool cards: amber accent for cost-bearing tools, neutral for free.
 - Cost badge: lamports formatted as SOL with 4 decimals + SOL, pill-style.
4. Persistence: panel state (collapsed/expanded, scroll position) persists across route changes via localStorage.
5. Mock data file for development: src/components/AuraConversation.mock.ts with 8 sample messages covering all four types, including a tool_call with cost + tx_signature. Used until T1's WS broadcaster is live.
6. Build clean, no new ESLint warnings.

End-of-block validation: live messages from T1's voice agent appear in the panel. Tool calls render correctly. Explorer links work.

P11: Run-through 1 + Hotfix (T4 leads, all participate, 30min)

Tool: terminal **Model:** Claude Sonnet 4.6 for hotfixes only

Execution, not generation. Two run-throughs of the full pitch with the live system.

15:00 --- Run-through 1 (full):

- T4 holds the mic, T3 operates dashboard, T1 sits at agent terminal, T2 at fleet terminal.
- Run the scripted three-act exchange from docs/pitch.md word-for-word.
- Time it. Record EVERY glitch: latency spikes, misrecognised utterances, tool failures, video stutters, dashboard rendering issues, voice level drift.
- Triage: glitches fixable in <5 min get fixed now. Glitches taking longer go to run_sheet.md as a Plan B and are accepted.

Common likely fixes:

- Tighten the confirmation classifier regex (T1).
- Tune system prompt to stop Aura from rambling (T1).
- Hardcode demo-bag event IDs into a pinned context message so Aura always has them available (T1).
- Lower TTS first-chunk threshold for snappier replies (T1).
- Bump video volume on the dashboard or mute entirely (T3 --- mute is safer; videos have no audio anyway).
- Fix WS auto-reconnect if a stream drops (T3 or T2).

15:15 --- hotfix window (15 min). Apply <5 min fixes. NO new features.

7 Phase 6: Saturday 15:30–15:45 — Dress Rehearsal

P12: Dress Rehearsal (all, 15min)

Tool: terminal **Model:** —

15:30 --- Run-through 2 (dress rehearsal):

- Same script. This IS the rehearsal --- whatever works here is what you ship.
- If it goes well, you are done.
- If it goes badly: there is no run-through 3. Commit to whatever works at 15:30 and own it.

15:40 --- micro-fixes only (no new code, only:

- Adjust audio levels.
- Reload a stale browser tab.
- Restart a misbehaving bridge process.
- Pre-load demo bag events into the visible compliance log.

15:45 --- HARD FREEZE. Close all editors. No code changes regardless of what's broken. Plug into projector. Audio + mic check. Dashboard windows arranged on factory floor view. Both team members rehearse the click choreography one more time silently.

15:55 --- breathe. T4 reviews the pitch script one final time mentally. Everyone else stays away from the laptop.

8 Appendix A: Model Selection Rationale

Task	Model	Why
P0.1 voice hello-world	Claude Sonnet 4.6	Standard integration code.
P0.2 dataset prep	Claude Sonnet 4.6	Schema reading + summarising.
P0.3 dashboard scaffold	Claude Sonnet 4.6	Toolchain bootstrap; no novelty.
P1 voice loop port	Claude Sonnet 4.6	Routine.
P2 VideoSource	Claude Opus 4.7	Timestamp alignment + cross-rate resampling are fiddly; Opus’s reasoning is worth it.
P3 factory floor view	Claude Sonnet 4.6	React + Tailwind boilerplate.
P4 robot registration	Claude Sonnet 4.6	Anchor/solana-py boilerplate.
P5 fleet-aware tools	Claude Sonnet 4.6	Five small wrappers around documented endpoints.
P6 multi-bridge runner	Claude Sonnet 4.6	Process orchestration; well-trodden territory.
P7 per-robot view	Claude Sonnet 4.6	Component composition.
P8 pitch + run sheet	Claude Opus 4.7	Writing quality matters; Opus’s prose lands the partners better.
P9 voice + tools wiring	Claude Opus 4.7	The hardest reasoning task — confirmation flow, async dispatch, error paths, prompt design.
P10 transcript panel	Claude Sonnet 4.6	WS subscriber + render.
Aura’s runtime reasoning model	GPT-4o	OpenAI sponsor credits; mature tool-use streaming; lower latency than Claude in voice loop.
Confirmation classifier	Claude Haiku 4.5	One-word classification on a 1s budget. Right-sized.

9 Appendix B: Scope Cuts & Anti-goals

What Aura explicitly does NOT do (saying so in the pitch is a strength):

- No live camera, no vision tool. Aura is voice + telemetry only. (Vision is the obvious v2.)
- No new Solana instructions. Every on-chain action uses Auxin’s existing program.
- No modifications to Auxin’s bridge except adding the **VideoSource** class.
- Three robots, not N. (Three is enough; more is theatre.)
- No persistent conversation memory across restarts.
- No streaming voice cloning or custom voice. Hardcoded ElevenLabs voice ID.
- No self-hosted Whisper. OpenAI API only.
- No real-time video sync to telemetry — videos loop independently, 1-2s drift is fine.

Triage rules under time pressure:

- T1 behind at 14:30: drop the proactive watcher. Reactive-only is fine.
- T1 behind at 15:15: drop the confirmation flow. Major demo cost — avoid this scenario.
- T2 behind at 14:00: ship with 2 robots not 3. Adjust pitch script.
- T3 behind at 14:30: ship factory floor view only; demo by hovering tiles, do not click through.
- T3 behind at 15:00: skip the persistent transcript panel; demo via terminal stdout.
- Auxin breaks at the venue: pre-record a successful end-to-end run Friday night. Mention it as “here’s a clip from our Friday integration test in case the venue WiFi gives us trouble live” — credible, professional move.
- Three bridges OOM the laptop: drop privacy providers to direct, kill Prometheus, set `rate_hz=5`. If still OOM: 2 robots.

The single most important rule: after 15:45, no code changes. Whatever works at 15:45 is what you demo. Late fixes break demos far more often than they save them.